🧠 GENAI WORKSHOP — DAY 3 NOTES

Topic: RAG with CODE + PDF (Student Point of View)

1️⃣ What you already know before today

From previous classes, you understood that:

- An **LLM (Gemini / GPT)** is *not a database*
- LLMs:
  - don't remember my files
  - don't know my codebase
  - hallucinate if context is missing
- **RAG** solves this by:
  - Storing knowledge outside the LLM
  - Retrieving relevant parts
  - Giving that context to the LLM

📌 Important realization
LLM answers ≠ correct answers
Context-based answers = reliable answers

2️⃣ What problem are we solving today?

Until now, our RAG system learned from **one Python file**.

That is **not realistic**.

In real systems:

- Knowledge lives in **many files**
- Knowledge is in **different formats**
  - code files (.py)

- documents (.pdf, .txt)
- guidelines
- internal docs

❓ Real question

Can one RAG system learn from **both code and documents**?

👉 **Yes — but only if we design it properly.**

3️⃣ New goal for today (very clear)

Build a RAG system that can answer questions using:

- Multiple Python files
- One document file (PDF / TXT)

Without changing how the user asks questions.

Same /ask endpoint.
Smarter backend.

4️⃣ Why CODE + PDF RAG is difficult (important)

🧩 Code is structured

- functions
- logic
- indentation
- keywords

📄 PDFs are unstructured

- headers
- footers
- page breaks

- noisy text

If we treat both the same:

❌ retrieval becomes weak
❌ answers become vague
❌ confidence drops

📌 This is why many RAG demos fail in production.

5️⃣ Core idea I learned today

🧠 RAG Pipeline (mental model)

```
Documents (code + pdf)
      ↓
Chunking (split intelligently)
      ↓

Embeddings (numbers)
      ↓
Vector Store (FAISS)
      ↓
Search (similarity)
      ↓
LLM (final answer)
```

⚠️ **LLM is the LAST step**
⚠️ Most mistakes happen BEFORE the LLM

6️⃣ What “multi-source ingestion” means

Instead of reading one file:

open("utils.py")

We now:

- scan a folder
- read **every file**
- tag each chunk with:
  - source file name
  - content type

Example chunk memory:

```
Chunk:
"def login(username, password)..."

Metadata:
source = auth.py
type = code
```

This makes answers **traceable**.

7️⃣ What good RAG answers look like (student clarity)

☑️ Good answer

- Comes directly from retrieved context
- Mentions logic present in files
- Says “Not found” when missing

❌ Bad answer

- Sounds smart but unsupported
- Uses general AI knowledge
- Ignores document content

📌 If the answer doesn’t come from retrieval, **RAG failed**.

8️⃣ Why we say “Not found in context”

This is **not a weakness**.

It is a **feature**.

It means:

- Retrieval is honest
- System is safe
- No hallucination

9️⃣ What confidence in RAG actually means

Confidence is NOT:

❌ memorizing theory
❌ copying code

Confidence IS:

✅ knowing where RAG breaks
✅ seeing wrong retrieval
✅ fixing chunking
✅ debugging similarity search

🧠 One-line takeaway (for revision)

RAG is not about the LLM.
RAG is about controlling knowledge flow.

RAG with CODE + PDF (PRACTICAL IMPLEMENTATION)

0️⃣ What we are building today (CRYSTAL CLEAR)

By the end of this class, **I will have**:

- A FastAPI app
- A RAG system that learns from:
  - Python code files (.py)
  - PDF documents (.pdf)
- One /ask endpoint
- Zero hallucinations
- Clear test cases in /docs

If an answer is not present 👉 **system must say**

"Not found in context"

1️⃣ FINAL TECH STACK (LOCK THIS IN YOUR HEAD)

Layer	Tool	Why
Embeddings	SentenceTransformers	Local, free, stable
Vector DB	FAISS	Fast similarity search
LLM	Gemini 2.5 Flash	Reasoning, cheap
Backend	FastAPI	Clean API
Docs Parsing	PyPDF	Read PDFs

⚠️ **Important rule**
LLM is **NOT** used for memory.
LLM is **ONLY** used for final answer generation.

2️⃣ PROJECT STRUCTURE (DO NOT CHANGE)

```
day3-rag/
│
├── main.py
├── ingest.py
├── vector_store.py
│
├── documents/
│   ├── utils.py
│   ├── auth.py
│   └── rules.pdf
│
├── requirements.txt
├── .env
└── README.md   (optional)
```

📌 **Golden Rule**
If a file is not inside documents/
👉 RAG **does not know it exists**

3️⃣ requirements.txt (INSTALL EVERYTHING ONCE)

```
fastapi
uvicorn
faiss-cpu
numpy
python-dotenv
sentence-transformers
google-genai
pypdf
```

Why each library exists

- sentence-transformers → embeddings
- faiss-cpu → similarity search
- pypdf → extract text from PDFs
- google-genai → Gemini 2.5 Flash
- fastapi + uvicorn → API server

4️⃣ .env FILE (VERY IMPORTANT)

```
GEMINI_API_KEY=your_real_key_here
```

⚠️ NEVER upload this to GitHub
⚠️ NEVER show this on screen

5️⃣ SOURCE FILES (WHAT RAG LEARNS FROM)

📄 documents/utils.py

```
def helper():
  print("Utility helper function")
```

📄 documents/auth.py

```
def login(username, password):
  hardcoded_password = "admin123"
  if password == hardcoded_password:
    return "Login successful"
  return "Login failed"
```

📄 documents/rules.pdf

```
PDF content (example):

Security Rules:
- Passwords must not be hardcoded
- Authentication should use hashing
- OAuth is preferred for production systems
```

📌 This PDF is **knowledge**, not code.

6️⃣ ingest.py — READ EVERYTHING CAREFULLY

This file does **3 things**:

1. Reads code files
2. Reads PDFs
3. Converts everything to embeddings

📄 ingest.py

```
from sentence_transformers import SentenceTransformer
from pypdf import PdfReader
import os
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")


def read_py_file(path):
  with open(path, "r") as f:
    return f.read()


def read_pdf_file(path):
```

```
    reader = PdfReader(path)
    text = ""
    for page in reader.pages:
        text += page.extract_text()
    return text


def chunk_text(text, chunk_size=200):
    words = text.split()
    chunks = []

    for i in range(0, len(words), chunk_size):
        chunks.append(" ".join(words[i:i + chunk_size]))

    return chunks


def load_documents():
    base_path = "documents"
    all_chunks = []

    for file in os.listdir(base_path):
        full_path = os.path.join(base_path, file)

        if file.endswith(".py"):
            text = read_py_file(full_path)

        elif file.endswith(".pdf"):
            text = read_pdf_file(full_path)

        else:
            continue

        chunks = chunk_text(text)
        all_chunks.extend(chunks)

    embeddings = model.encode(
        all_chunks,
```

```
    convert_to_numpy=True,
    normalize_embeddings=True
  )

  return all_chunks, embeddings.astype("float32")
```

🧠 **Student insight**

PDFs and code are treated the same **after chunking**

7️⃣ vector_store.py — SIMPLE BUT IMPORTANT

```
import faiss

def build_index(vectors):
  dim = vectors.shape[1]
  index = faiss.IndexFlatL2(dim)
  index.add(vectors)
  return index
```

8️⃣ main.py — FULL RAG API

📄 main.py

```
import os
import numpy as np
from fastapi import FastAPI
from dotenv import load_dotenv
from google import genai
from sentence_transformers import SentenceTransformer

from ingest import load_documents
from vector_store import build_index
```

```
load_dotenv()

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

app = FastAPI()

# Load RAG memory
chunks, vectors = load_documents()
index = build_index(vectors)

embedder = SentenceTransformer("all-MiniLM-L6-v2")


@app.post("/ask")
def ask(question: str):
  query_vector = embedder.encode(
    question,
    convert_to_numpy=True,
    normalize_embeddings=True
  ).astype("float32")

  _, indices = index.search(
    np.array([query_vector]),
    k=3
  )

  context = "\n".join([chunks[i] for i in indices[0]])

  prompt = f"""
Answer ONLY using the context below.
If the answer is not present, say "Not found in context".

Context:
{context}

Question:
{question}
```

```
"""

  response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
  )

  return {"answer": response.text}
```

9 HOW TO RUN (STEP BY STEP)

1 Create virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

2 Install dependencies

```
pip install -r requirements.txt
```

3 Start server

```
python3 -m uvicorn main:app --reload
```

You should see:

Uvicorn running on http://127.0.0.1:8000

10 TESTING IN FASTAPI DOCS (VERY IMPORTANT)

Open:

👉 http://127.0.0.1:8000/docs

☑ TEST CASE 1 — Code Question

```
{
 "question": "Why is the login function insecure?"
}
```

Expected:

- Mentions hardcoded password
- Quotes logic from auth.py

☑ TEST CASE 2 — PDF Question

```
{
 "question": "What are the security rules for passwords?"
}
```

Expected:

- Mentions rules from PDF

❌ TEST CASE 3 — Missing Info

```
{
 "question": "Does this system use JWT authentication?"
}
```

Expected:

"Not found in context"

✅ This proves **NO hallucination**

🧠 COMMON STUDENT CONFUSIONS (CLEARED)

❓ Why embeddings again at query time?

Because questions must be converted to numbers.

❓ Why FAISS?

Because LLM cannot search vectors.

❓ Why not store everything in prompt?

Because token limits will kill your app.

FINAL REALIZATION

RAG is not AI magic.
RAG is disciplined engineering.

You now:

- control memory
- control truth
- control hallucinations